

Informatik 2

Kapitel 6 – Ausnahmen

Inhaltsverzeichnis

6.1 Einleitung	2
6.2 Fallstudie grafischer Editor: Ausnahmen	6
6.3 Grundlagen zu Ausnahmen	11
6.3.1 Ausnahmen werfen (throw)	11
6.3.2 Ausnahmen weiterreichen (throws) und abfangen (try und catch)	12
6.3.3 Eigene Ausnahmeklassen implementieren	15

6.1 Einleitung

Worum wir uns bisher noch gar nicht gekümmert haben, ist die Behandlung von Fehlern in Java. Solche Fehler können zum Beispiel auftreten, wenn man an eine Methode unzulässige Werte übergibt, wie zum Beispiel in der ersten der beiden folgenden Anweisungen:

```
int n = Integer.parseInt("a");
System.out.println(2 * n);
```

Mit der Methode `parseInt` lässt sich ein String, die eine ganze Zahl repräsentiert (also ein bestimmtes Format aus optionalem Vorzeichen und Ziffern hat), in einen `int`-Wert umwandeln.

Was passiert aber, wenn man wie oben einen String übergibt, die keine ganze Zahl repräsentiert?

Dann kommt es zum Programmabbruch und zu einer Fehlerausgabe auf der Kommandozeile, die etwa wie folgt aussieht:

```
Exception in thread "main" java.lang.NumberFormatException: For input
string: "a"
at java.base/java.lang.NumberFormatException.forInputString(
    NumberFormatException.java:65)
at java.base/java.lang.Integer.parseInt(Integer.java:652)
at java.base/java.lang.Integer.parseInt(Integer.java:770)
at Test.main(Test.java:7)
```

Hier scheint es zu einer Ausnahme namens `NumberFormatException` gekommen zu sein. Man sagt, dass diese Ausnahme von `parseInt` **geworfen wurde**. Ähnlich sieht es aus, wenn man wie folgt mit einem `Scanner`-Objekt über Kommandozeile eine ganze Zahl vom Benutzer einliest:

```
Scanner userInput = new Scanner(System.in);
int n = userInput.nextInt();
```

Gibt der Benutzer eine Zeichenfolge ein, die nicht als ganze Zahl interpretiert werden kann, so kommt es wieder zu einem Programmabbruch – diesmal wird eine `InputMismatchException` geworfen.

In der Programmiersprache C war das noch anders. Hier konnten wir wie folgt eine ganze Zahl vom Benutzer einlesen:

```
int n;
int status = scanf("%i", &n);
if (status != 1)
    printf("Ungültige Eingabe");
```

In C werden Fehlerfälle über den Rückgabewert von Funktionen angezeigt. So hat obiger Aufruf von `scanf` einen Rückgabewert ungleich 1, falls der Benutzer eine Zeichenfolge eingibt, die nicht als ganze Zahl interpretiert werden kann. Diesen Rückgabewert können wir abfragen und entsprechend eine Fehlerbehandlung durchführen – zum Beispiel eine Ausgabe wie `"Ungültige Eingabe"` an den Benutzer.

In Java werden Fehler bei Aufruf einer Methode nun nicht durch einen spezifischen Rückgabewert angezeigt, sondern durch das Erzeugen einer Ausnahme, die erstmal zu einem Programmabbruch führt. Das hat den Vorteil, dass der Entwickler sich um eine Fehlerbehandlung kümmern **muss**, wenn das Auftreten einer Ausnahme möglich ist und das Programm deshalb nicht abstürzen soll.

Nun wäre es schön zu wissen, wie wir diesen Programmabbruch vermeiden könnten, um stattdessen eine geordnete Fehlerbehandlung zu machen.

Dafür kann man geworfene Ausnahmen **abfangen**. Das sieht im Beispiel wie folgt aus:

```
try {
    int n = Integer.parseInt("a");
    System.out.println(2 * n);
} catch (NumberFormatException e) {
    System.out.println(e.getMessage());
}
```

Die eine Ausnahme werfende Methode steht in einem sogenannten **try-Block**. In diesem **try-Block** stehen im Allgemeinen noch weitere Anweisungen nach dem kritischen Methodenaufruf, die im Ausnahmefall nicht sinnvoll ausgeführt werden können. Wird eine Ausnahme geworfen, werden diese Anweisungen übersprungen (ignoriert).

Danach erfolgt die Fehlerbehandlung in einem sogenannten **catch-Block**. Hierfür kann man auf die geworfene Ausnahme `e` zugreifen und zum Beispiel auf die in ihr gespeicherten Fehlernachricht mit `getMessage` zugreifen und diese ausgeben. **Ausnahmen sind also Objekte** mit Informationen über die Art und Ursache von Fehlern, und erlauben insbesondere die Festlegung und Ausgabe spezieller **Fehlermeldungen**. Nach dem **catch-Block** kommen dann alle Anweisungen, die immer ausgeführt werden sollen, egal ob eine Ausnahme geworfen wurde oder nicht.

Um eine solche Fehlerbehandlung machen zu können, muss man natürlich wissen, welche Methode welche Ausnahmen in welchen Fällen wirft. Diese Informationen findet man alle in der API. Zum Beispiel hat die `parseInt`-Methode den folgenden Methodenkopf:

```
public static int parseInt(String s) throws NumberFormatException
```

Wir entnehmen die Information, welche Ausnahme geworfen wird, direkt dem Methodenkopf. Nach dem Schlüsselwort `throws` kommt eine Liste von Ausnahmen (durch Kommas getrennt), die die Methode in bestimmten Fehlerfällen wirft. In der Beschreibung der `parseInt`-Methode finden wir dann Informationen über diese Fehlerfälle:

Throws:

`NumberFormatException` - if the string does not contain a parsable integer.

Erfolgt der Aufruf einer Methode, die eine Ausnahme werfen kann, in einer zweiten Methode, so können wir in der zweiten Methode auch auf das Abfangen der Ausnahme verzichten und diese stattdessen **weiterreichen** (an einen Aufrufer der zweiten Methode). Um zu sehen, wie das geht und warum es Sinn machen kann, so vorzugehen, schauen wir uns im nächsten Unterkapitel ein ausführliches Beispiel an. Dazu erweitern wir dort unsere Fallstudie des grafischen Editors.

Sehen wir in die API, stellen wir fest, dass Ausnahmen in Java (wie auch Collections) eine Vererbungshierarchie bilden. In Abbildung 1 sehen Sie einen Ausschnitt dieser Vererbungshierarchie. Alle Ausnahmeklassen erben also von der Oberklasse Throwable. Je nach Unterklasse liegen verschiedene Arten von Fehlern vor. Die gemeinsame Oberklasse Throwable stellt allen Ausnahmeklassen unter anderem folgende Funktionalitäten zur Verfügung:

- **Festlegung und Abfrage einer Fehlernachricht zur Beschreibung des Fehlers.**
Die Fehlernachricht kann über den Konstruktor gesetzt und mit der Methode `getMessage` abgefragt werden.
- **Festlegung und Abfrage einer anderen Ausnahme als Fehlergrund.**
Der Fehlergrund kann über den Konstruktor gesetzt und mit der Methode `getCause` abgefragt werden. Damit können Ausnahmen verpackt in andere Ausnahmen durchgereicht werden.
- **Abfrage der Methodenaufrufe, die zur Ausnahme geführt haben.**
Des Stack der Methodenaufrufe bis zum Auftreten der Ausnahme kann mit `printStackTrace` auf `System.err` (das ist die Standardfehlerausgabe auf Kommandozeile) ausgegeben werden.

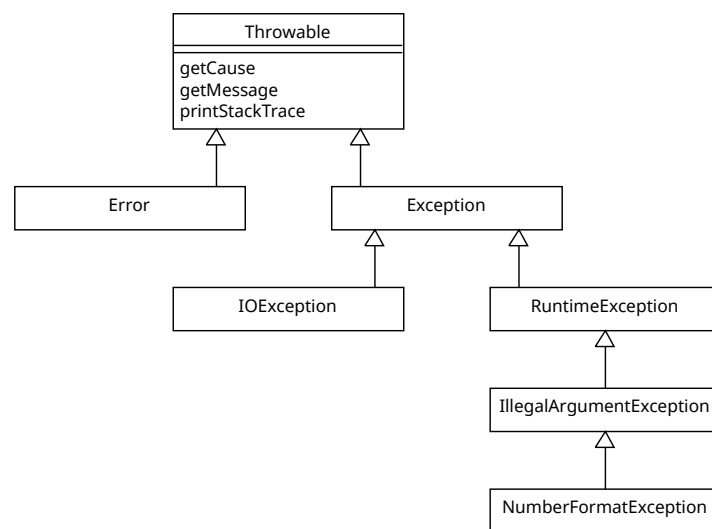


Abbildung 1: Ausschnitt der API-Vererbungshierarchie für Ausnahmen.

Die Klasse `Error` ist eine Klasse für schwerwiegende Fehler, die niemals auftreten sollten und auf die nur **mit Programmabbruch** reagiert werden kann, da man ein Programm danach üblicherweise nicht sinnvoll fortsetzen kann. Deshalb kann ein `Error` auch nicht abgefangen werden. Zu einem `Error` kommt es zum Beispiel, wenn nicht mehr genügend Speicherplatz vorhanden ist (`OutOfMemoryError`). Die meisten solchen schwerwiegenden Fehler gehen auf Programmierfehler zurück. Kommt es zu einem `Error`, so empfiehlt es sich, den Quellcode nochmal genau anzuschauen, wo etwas schief läuft. Ansonsten werden wir diese Klasse in der Vorlesung nicht weiter benötigen.

Die Klasse `Exception` wird für weniger schwerwiegende Fehler benutzt, **nach deren geeigneter Behandlung das Programm fortgesetzt werden kann**. Solche Fehler können wir also abfan-

gen. Es gibt viele verschiedene Exception-Unterklassen, mit denen wir unterschiedliche Arten von Fehlern anzeigen können. Später lernen wir zum Beispiel die Fehlerklasse `IOException` für Ausnahmen durch Eingabe/Ausgabe-Fehler bei Dateizugriff kennen.

Eine besondere Unterklasse von `Exception` ist die Klasse `RuntimeException`. Alle Ausnahmen vom Typ `RuntimeException` nennt man **ungeprüfte Ausnahmen**.

Definition 6.1: Ungeprüfte Ausnahme

Ausnahmen, die **nicht behandelt werden müssen**, heißen **ungeprüfte Ausnahmen** / **unchecked exceptions**.

Die Klasse `RuntimeException` ist die gemeinsame Oberklasse aller ungeprüfter Ausnahmen.

Ungeprüfte Ausnahmen müssen nicht (aber können) im Methodenkopf einer Methode angezeigt werden.

Ein Beispiel für eine ungeprüfte Ausnahme ist die `NumberFormatException`. Diese wird unter anderem, wie wir bereits in einem Beispiel gesehen haben, von der `parseInt`-Methode der Klasse `Integer` geworfen und von dieser auch im Methodenkopf angezeigt.

Ein Beispiel für eine Methode, die eine ungeprüfte Ausnahme nicht in ihrem Methodenkopf anzeigt, ist die `charAt`-Methode der Klasse `String`: Diese wirft eine `IndexOutOfBoundsException`, falls der übergebene Index nicht zu einem Zeichen des Strings gehört, also negativ oder nicht kleiner als die Länge des Strings ist.

Diese Ausnahmen sind ungeprüft, weil man sie in der Regel durch sorgfältiges Programmieren vermeiden kann. Zum Beispiel lässt sich beim Durchlaufen der Zeichen eines Strings mit `charAt` leicht eine `IndexOutOfBoundsException` vermeiden, wenn man auf die korrekten Grenzen für den Index achtet.

Ein weiteres gutes Beispiel für eine ungeprüfte Ausnahme, die sich durch sorgfältige Programmierung vermeiden lässt, ist die `NullPointerException`: Diese wird erzeugt beim Aufruf einer Methode für eine `null`-Referenz.

Ungeprüfte Ausnahmen deuten in der Regel auf Programmierfehler hin, können also durch sorgfältige Programmierung vermieden werden.

Deshalb fängt man sie auch nicht zwingend ab – ein durch eine ungeprüfte Ausnahme ausgelöster Programmabbruch zeigt dem Entwickler an, dass der Code verändert werden muss!

Definition 6.2: Geprüfte Ausnahme

Ausnahmen, **die abgefangen (behandelt) werden müssen**, heißen **geprüfte Ausnahmen / checked exceptions**.

Alle Unterklassen der Klasse `Exception`, die nicht vom Typ `RuntimeException` sind, sind geprüfte Ausnahmen.

Geprüfte Ausnahmen müssen im Methodenkopf einer Methode angezeigt werden.

Demgegenüber werden geprüfte Ausnahmen für eingeplantes (fehlerhaftes) Verhalten einer Anwendung benutzt, das nicht durch Programmierung verhindert werden kann.

Darunter fällt insbesondere fehlerhaftes Benutzerverhalten, wie zum Beispiel ungültige Benutzereingaben. In solchen Fällen soll eine Anwendung ein definiertes Verhalten haben und natürlich nicht mit einem Programmabbruch reagieren – deshalb fängt man solche Fehler ab.

6.2 Fallstudie grafischer Editor: Ausnahmen

Auch bei der Verwendung von Methoden zur Verwaltung der Daten einer Anwendung (in eigenen Datenklassen) kann es zu Fehlern kommen. Oft unterliegen die erlaubten Daten einer Anwendung bestimmten Einschränkungen. So können zum Beispiel nicht alle Werte des gewählten Datentyps eines Attributs erlaubt sein. Das trifft auch auf die Datenklassen unseres grafischen Editors zu. Hier sind für die Koordinaten in der Klasse `Point` nur nicht-negative Werte erlaubt, weil nur solche Koordinaten auf der Zeichenfläche darstellbar sind (siehe Exkurs zu Zeichenflächen in Java in Kapitel 2). Dasselbe trifft auf den Radius eines Kreises, sowie auf Länge und Breite eines Rechtecks zu – auch diese dürfen nicht negativ sein. Solche Einschränkungen der Attributwerte einer Datenklasse nennt man **Datenstrukturinvarianten**. Erfüllt ein Attributwert eine vorgegebene Datenstrukturinvariante nicht, so nennen wir ihn **ungültig**, sonst **gültig**. Unsere erste Implementierung in Kapitel 2 war (wie Sie sich vielleicht erinnern) noch in der Hinsicht vereinfacht, dass wir bisher keine solchen Datenstrukturinvarianten berücksichtigt haben. Das bedeutet, dass bisher zum Beispiel in Settern auch ungültige Werte gesetzt werden können. Das wollen wir nun ändern.

Merke 6.3: Gültige und ungültige Werte

Eine Eigenschaft eines Attributwerts, die immer zutreffen soll, nennt man **Datenstrukturinvariante**. Attributwerte, die alle Datenstrukturinvarianten erfüllen, nennt man **gültig**. Attributwerte, die eine Datenstrukturinvariante nicht erfüllen, nennt man **ungültig**.

Wir wollen ab jetzt in unserer Implementierung dafür sorgen, dass nur gültige Werte gesetzt werden können. Das Setzen ungültiger Werte hingegen soll zu Ausnahmen führen, für die wir an geeigneter Stelle eine Fehlerbehandlung durchführen. Dazu nehmen wir folgende Änderungen an der Implementierung vor:

- Die Überprüfung, ob ein Wert ein ungültiger Wert bezüglich einer Datenstrukturinvariante ist, lagern wir in eine separate Check-Methode aus. Das ist eine Klassenmethode, da mit ihr keine Objektattribute verwaltet werden, sondern eine Datenstrukturinvariante der Klasse getestet wird.
- In einer Set-Methode überprüfen wir zuerst mit Check-Methoden, ob ein ungültiger Wert übergeben wurde. Ist der Wert ungültig, werfen wir eine Ausnahme, ansonsten setzen wir den Wert.
- Für die Ausnahme legen wir eine eigene Ausnahmeklasse an. Hierbei müssen wir uns jeweils entscheiden, ob diese geprüft oder ungeprüft sein soll.

Das sieht für die Klasse `Point` wie folgt aus:

```
public class Point {

    private int x;
    private int y;

    public Point(int x, int y) throws IllegalArgumentException {
        setX(x);
        setY(y);
    }

    public Point(Point original) throws IllegalArgumentException {
        setX(original.getX());
        setY(original.getY());
    }

    private static boolean checkNonNegative(int value) {
        return (value >= 0);
    }

    public int getX() {
        return x;
    }

    public void setX(int x) throws IllegalArgumentException {
        if (!checkNonNegative(x))
            throw new IllegalArgumentException("x coordinate invalid
            ", x + "");
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) throws IllegalArgumentException {
        if (!checkNonNegative(y))
            throw new IllegalArgumentException("y coordinate invalid
            ", y + "");
        this.y = y;
    }
}
```

```

    }

    @Override
    public String toString() {
        return "(" + getX() + ", " + getY() + ")";
    }
}

```

Die statische Check-Methode heißt `checkNonNegative`. Ihr Name ist so gewählt, dass man erkennt, welche Eigenschaft sie testet. Sie testet, ob eine übergebene ganze Zahl `value` nicht-negativ ist.

Merke 6.4: Check-Methoden

Eine **Check-Methode** ist eine statische Methode, die testet ob ein übergebener Wert bezüglich einer Datenstrukturinvariante gültig ist. Ist der Wert gültig, liefert sie `true`, sonst `false`.

In den beiden Set-Methoden überprüfen wir mit `checkNonNegative`, ob der übergebene Wert nicht-negativ ist. Ist dies der Fall, setzen wir den Wert. Ansonsten werfen wir eine Ausnahme vom Typ `IllegalDimensionException`. Dies geschieht mit `throw` gefolgt von einem Konstruktoraufruf. An den Konstruktor übergeben wir eine Fehlermeldung und den ungültigen Wert. Diese können an jeder Stelle abgefragt werden, an der die Ausnahme abgefangen wird. Wird eine Ausnahme geworfen, werden alle nachfolgenden Anweisungen übersprungen. In diesem Fall wird also kein Wert für `x` gesetzt. Der Kopf der Set-Methoden zeigt mit

```
throws IllegalDimensionException
```

an, dass sie eine solche Ausnahme werfen können. Um die Ausnahme muss man sich kümmern, sobald man eine Set-Methode aufruft.

Merke 6.5: Set-Methoden und Ausnahmen

In Set-Methoden überprüfen wir mit Check-Methoden, ob der übergebene Attributwert gültig oder ungültig ist. Ist er ungültig, so wirft die **Set-Methode** eine Ausnahme. Nur wenn der Wert gültig ist, wird er gesetzt.

In den Konstruktoren rufen wir nun die Set-Methoden auf und dabei kann jetzt die neue Ausnahme geworfen werden. Hier können wir die Ausnahme nun entweder abfangen und eine Fehlerbehandlung durchführen oder diese an Aufrufer des Konstruktors weiterreichen. Wir entscheiden uns hier dafür, die Ausnahme weiterzureichen. Dafür müssen wir nichts weiter tun, als dies im Kopf mit

```
throws IllegalDimensionException
```

anzuzeigen. Um die Ausnahme muss man sich dann erst kümmern, sobald man den Konstruktor aufruft. Dadurch können wir die Fehlerbehandlung abhängig vom Ort des Konstruktoraufrufs individuell gestalten. Insbesondere kann die Fehlerbehandlung so später auch abhängig von der Anwendung erfolgen, in der man die Klasse `Point` benutzt. Würden wir die Ausnahme schon im

Konstruktor mit zugehöriger Fehlerbehandlung abfangen, würde diese Fehlerbehandlung einheitlich für alle Konstruktoraufrufe und Anwendungen gelten.

Merke 6.6: Konstruktoren und Ausnahmen

Konstruktoren reichen von Set-Methoden geworfene Ausnahmen weiter, um eine individuelle Fehlerbehandlung zu ermöglichen.

Die neue Ausnahmeklasse heißt `IllegalDimensionException`. Ihr Name ist so gewählt, dass man erkennt, für welche Daten ein ungültiger Wert übergeben wurde. Eine mögliche Implementierung sieht wie folgt aus:

```
public class IllegalDimensionException extends RuntimeException {

    private static String format = "The value must be non-negative";
    private String value;

    public IllegalDimensionException(String message, String value) {
        super(message);
        this.value = value;
    }

    public static String getFormat() {
        return format;
    }

    public String getValue() {
        return value;
    }
}
```

Wir legen unsere neue Ausnahmeklasse also als Unterklasse der API-Klasse `RuntimeException` an. Damit ist diese eine ungeprüfte Ausnahme. Das ist sinnvoll, falls wir alle Benutzereingaben oder -interaktionen verhindern, die in negativen Werten für Koordinaten von Punkten resultieren. Das kann zum Beispiel der Fall sein, wenn der Benutzer versucht, Formen mit der Maus über den linken oder oberen Rand der Zeichenfläche zu verschieben. Das haben wir in unserer Implementierung zum Beispiel bereits in der Implementierung der move-Methoden berücksichtigt und kann deshalb nicht vorkommen.

Als Unterklasse von `RuntimeException` können wir alle Funktionalitäten der Oberklasse benutzen. Insbesondere können wir

- den Konstruktor der Oberklasse aufrufen und an diesen eine individuelle Fehlernachricht übergeben (`super(message)`), und
- diese Fehlernachricht bei Bedarf mit der `getMessage`-Methode abfragen.

Zudem ergänzen wir zwei zusätzliche Attribute:

- In `format` speichern wir Informationen über die Attributeigenschaft, die erfüllt sein muss, damit der Attributwert gültig ist.

- In `value` speichern wir Informationen über den ungültigen Attributwert.

Beide Informationen können wir bei Bedarf mit Get-Methoden abfragen.

Übung: Ergänzen Sie nun nach demselben Muster auch die Klassen `Form`, `Circle` und `Rectangle` um Ausnahmen für ungültige Werte. Berücksichtigen Sie dazu die folgenden zusätzlichen Datenstrukturinvarianten:

- Der Radius eines Kreises darf nur nicht-negative Werte annehmen.
- Der Länge und Breite eines Rechtecks darf nur nicht-negative Werte annehmen.

Werfen Sie bei ungültigen Werten in allen Fällen eine `IllegalDimensionException` mit passender Fehlermeldung und passenden Zusatzinformationen.

Nach diesen Regeln können wir allgemein bei der Implementierung von Datenklassen mit Ausnahmen vorgehen. Das bedeutet, es ist grundsätzlich sinnvoll, dass Set-Methoden und Konstruktoren und gegebenenfalls weitere Methoden einer Datenklasse alle Ausnahmen weiterreichen. Zum Beispiel kann es verschiedene Benutzerschnittstellen geben, über die Benutzereingaben erfolgen können, wie eine Webseite für den Kunden und verschiedene Eingabefenster verschiedener Programme für Mitarbeiter der Firma, die Kundendaten bearbeiten und verwalten. In diesen Fällen sind gegebenenfalls unterschiedliche Fehlerbehandlungen erwünscht. Insbesondere sollte der Benutzer jeweils eine geeignete Fehlermeldung bekommen, zum Beispiel als Warntext oder über einen separaten Warndialog. Dafür implementiert man die Fehlerbehandlung am besten in der jeweiligen Benutzerschnittstelle und reicht die Ausnahme bis dorthin weiter. Andernfalls müssten wir in den Set-Methoden oder den Konstruktoren für alle vorhandenen Benutzerschnittstellen eine individuelle Fehlerbehandlung verwalten. Da es oft passiert, dass Benutzerschnittstellen wegfallen, neu hinzukommen oder sich ändern, ist ein solches Vorgehen nicht sinnvoll. Am liebsten wollen wir aus Gründen der Wartbarkeit und Übersichtlichkeit nichts mehr an Set-Methoden und Konstruktoren anpassen müssen, wenn sich Benutzerschnittstellen ändern!

Da wir bisher noch keine grafische Benutzerschnittstelle programmiert haben, wollen wir nun zur Übung geworfene Ausnahmen in der `main`-Methode abfangen. Dazu erzeugen wir `Form`-Objekte und nehmen diese mit der Methode `addForm` in den zugehörigen Container auf. Da der `Form`-Konstruktor eine `IllegalDimensionException`-Ausnahme auslösen kann, fangen wir diese in einem `try`-Block ab. Im `try`-Block befinden sich noch weitere Anweisungen, nämlich

- die Aufnahme des neu erstellten Objekts in den Container.
- und gegebenenfalls weitere Anweisungen zur Verwaltung des Containers.

```
FormContainer con = new FormContainer();
try {
    Circle c = new Circle(new Point(10, -10));
    con.addForm(c);
    con.addForm(new Rectangle(-50, 50, new Point(20, 75)));
    ...
} catch (IllegalDimensionException e) {
    System.out.println("Form nicht angelegt:" + e.getMessage());
    System.out.println("Wert:" + e.getValue());
    System.out.println("Format:" + e.getFormat());
}
System.out.println("Folgende Formen befinden sich im Container:");
```

```
con.forEach(System.out::println);
```

Kommt es beim `Circle`-Konstruktoraufzuruf zu einer Ausnahme, werden die Wertzuweisung an `c` und alle nachfolgenden Anweisungen im `try`-Block übersprungen. Dadurch entsteht keine Referenz auf das neu erstellte Objekt und der Container bleibt unverändert. Stattdessen erfolgt die Fehlerbehandlung im `catch`-Block. Diese besteht aus der Ausgabe einer passenden Fehlermeldung und weiterer nützlicher Informationen an den Benutzer.

Kommt es beim `Rectangle`-Konstruktoraufzuruf zu einer Ausnahme, werden `addForm` und alle nachfolgenden Anweisungen im `try`-Block nicht weiter ausgeführt. Auch hier bleibt der Container wieder unverändert.

6.3 Grundlagen zu Ausnahmen

Nach den einführenden Beispielen wollen wir nun die Grundlagen zu Ausnahmen allgemein besprechen:

- Das Werfen von Ausnahmen (`throw`).
- Das Anzeigen von Ausnahmen durch API-Methoden (`throws`).
- Das Weiterreichen von Ausnahmen (`throws`).
- Das Abfangen von Ausnahmen (`try` und `catch`).
- Die Implementierung eigener (geprüfter und ungeprüfter) Ausnahmen.

Hierbei lassen wir mit `finally` noch ein wichtiges Konzept vorläufig aus. Dieses holen wir nach, sobald wir weit genug sind, dafür ein sinnvolles Beispiel zu diskutieren.

6.3.1 Ausnahmen werfen (`throw`)

Möchte man in einer selbst programmierten Methode, wie zum Beispiel in `setX` der Klasse `Point`, für einen möglicherweise auftretenden Fehler eine Ausnahme erzeugen, so macht man das wie folgt:

Definition 6.7: Ausnahmen erzeugen in einer Methode / einem Konstruktor

Eine Ausnahme wird durch eine Anweisung der Form

```
throw new <Konstruktoraufzuruf>;
```

erzeugt.

- In der Regel wird an den Konstruktor zumindest eine spezifische Fehlermeldung übergeben.
- Diese Anweisung erzeugt ein Ausnahme-Objekt.
- Nachfolgender Code wird nicht mehr ausgeführt.

Damit der Code möglichst gut lesbar wird, möchte man dabei direkt am Namen der verwendeten Ausnahmeklasse die Art des Fehlers ablesen können. Dafür sucht man sich eine möglichst passende Ausnahmeklasse aus der API, oder noch besser, man implementiert eine neue spezifische eigene (geprüfte oder ungeprüfte) Ausnahmeklasse. Wird eine geprüfte Ausnahme erzeugt, so muss diese Ausnahme im Methodenkopf wie folgt möglichen Aufrufern angezeigt werden.

Definition 6.8: In einer Methode / einem Konstruktor erzeugte Ausnahmen anzeigen

In einer Methode erzeugte Ausnahmen werden Aufrufern der Methode angezeigt durch folgende Form des Methodenkopfes:

```
<Sichtbarkeit> [static] <Typ> <Methodenname>  
(Parameterliste) throws <Ausnahmeliste>
```

- <Ausnahmeliste> ist eine Liste von durch Komma getrennten Ausnahmen.
- In dieser Liste müssen alle geprüften Ausnahmen enthalten sein. Ungeprüfte Ausnahmen können enthalten sein. Damit wird jedem, der die Methode verwendet, angezeigt, dass diese eine bestimmte Ausnahme auslösen kann.

Auch Konstruktoren können Ausnahmen auf diese Art anzeigen.

Ein Entwickler, der eine Methode oder einen Konstruktor aufruft, muss somit nicht deren Implementierung kennen, sondern kann am Kopf ablesen, dass er sich gegebenenfalls um eine Ausnahme kümmern muss.

Übung: Recherchieren Sie Ihnen bisher bekannte Methoden der Java-API:

- Können diese Ausnahmen auslösen, und wenn ja, welche?
- Und sind dies ungeprüfte oder geprüfte Ausnahmen?

6.3.2 Ausnahmen weiterreichen (throws) und abfangen (try und catch)

Wenn man sich beim Aufruf einer Methode oder eines Konstruktors nicht um geworfene Ausnahmen kümmern möchte, kann man diese wie im Beispiel des `Point`-Konstruktors auf dieselbe Art und Weise weiterreichen wie man diese anzeigt.

Definition 6.9: Ausnahmen weiterreichen

Eine in einer Methode `outerMethod` durch Aufruf einer anderen Methode `innerMethod` geworfene Ausnahme `innerException` kann an Aufrufer von `outerMethod` weitergereicht werden durch folgende Form des Methodenkopfes:

```
<Sichtbarkeit> [static] <Typ> outerMethod  
(Parameterliste) throws innerException
```

- Auch Konstruktoren können Ausnahmen auf diese Art weiterreichen.
- Die `main`-Methode kann keine Ausnahmen weiterreichen.

Spätestens in der `main`-Methode muss also eine Behandlung geprüfter Ausnahmen erfolgen. Ungeprüfte Ausnahmen müssen nicht zwingend behandelt werden – in diesem Fall riskiert man einen Programmabbruch zur Laufzeit. Bei der Ausnahmebehandlung in Java spricht man auch vom **Abfangen von Ausnahmen**.

Definition 6.10: Ausnahmen abfangen

Eine von einer Methode geworfene Ausnahme wird wie folgt abgefangen:

```
try {  
    ...  
    <MethodeMitAusnahme>  
    <IgnorierteAnweisungen>  
} catch (<Ausnahmeklasse> e) {  
    <Fehlerbehandlung>  
}  
<GemeinsameFortsetzung>
```

- Die die Ausnahme auslösende Methode wird in einem sogenannten `try`-Block aufgerufen.
- Nach dem `try`-Block befindet sich ein `catch`-Block. Die geworfene Ausnahme muss vom Typ der im `catch`-Block für `e` verwendeten Ausnahmeklasse sein, damit diese dem `catch`-Block zugeordnet werden kann.
- Wirft der Methodenaufruf die Ausnahme, so werden alle nachfolgenden Anweisungen im `try`-Block übersprungen und der Code im `catch`-Block ausgeführt. Auch eine gegebenenfalls vorhandene Wertzuweisung des Rückgabewerts der Methode wird übersprungen.
- Im `catch`-Block steht der Code, der im Ausnahmefall ausgeführt werden soll. Dabei kann über die Referenz `e` auf das geworfene Ausnahmeobjekt zugegriffen werden.
- Anweisungen nach dem `catch`-Block werden immer ausgeführt, unabhängig davon, ob es zu einer Ausnahme gekommen ist, oder nicht.

Der `try`-Block sollte nach dem Methodenaufruf also genau diejenigen Anweisungen enthalten, die nur ausgeführt werden sollen oder können, wenn es nicht zur Ausnahme kommt. Kommt es in

unserem bereits betrachteten Beispiel

```
FormContainer con = new FormContainer();
try {
    Circle c = new Circle(new Point(10, -10));
    con.addForm(c);
    ...
} catch (IllegalDimensionException e) {
    System.out.println("Form nicht angelegt:" + e.getMessage());
    ...
}
```

beim Aufruf des `Circle`-Konstruktors in der Anweisung

```
Circle c = new Circle(...);
```

zu einer Ausnahme, wird damit die Wertzuweisung an `c` nicht mehr ausgeführt. Dadurch ist die nachfolgende Anweisung

```
con.addForm(c);
```

nicht sinnvoll ausführbar – sie steht deshalb mit im `try`-Block, damit sie im Ausnahmefall übersprungen wird.

Die im `catch`-Block für `e` verwendete Ausnahmeklasse muss nicht zwingend genau dem Typ der ausgelösten Ausnahme entsprechen, sondern kann auch eine Oberklasse davon sein. Theoretisch kann man deshalb als Ausnahmeklasse im `catch`-Block immer `Exception` verwenden, da dies die gemeinsame Oberklasse für alle Ausnahmen ist. Das ist aber schlechter Programmierstil, denn

- erstens wird das Programm so schlechter lesbar (man sieht dem `catch`-Block nicht mehr den genauen Fehlergrund an),
- und zweitens stehen dann im `catch`-Block auch nur Methoden der Klasse `Exception` für `e` zur Verfügung.

Als Best-Practice wollen wir uns deshalb daran halten, Ausnahmen immer so spezifisch wie möglich abzufangen.

In einem `try`-Block können auch **mehrere verschiedene Ausnahmen** auftreten. In diesem Fall schließen sich mehrere separate `catch`-Blöcke an den `try`-Block an – und zwar für jede Ausnahme einer:

```
try {
    <MehrereVerschiedeneAusnahmen>
} catch (<Ausnahmeklasse1> e) {
    ...
} catch (<Ausnahmeklasse2> e) {
    ...
}
...
```

Dies ermöglicht eine **individuelle Reaktion** auf jede Ausnahmeart. Ausgelöste Ausnahmen werden (von oben nach unten) im **ersten zu deren Typ passenden** `catch`-Block behandelt. Aus diesem Grund kommen `catch`-Blöcke zu spezielleren Ausnahmetypen vor `catch`-Blöcken zu allgemeineren Ausnahmetypen in der Vererbungshierarchie.

Falls mehrere Ausnahmen gleich behandelt werden sollen, so können diese gemeinsam in einem `catch`-Block abgefangen werden:

```
try {  
    <MehrereVerschiedeneAusnahmen>  
} catch (<Ausnahmeklasse1> | <Ausnahmeklasse2> e) {  
    <GemeinsameFehlerbehandlung>  
}  
...
```

Wir fassen zusammen: Kann eine Methode in Java fehlerhaftes Verhalten haben, so löst sie eine Ausnahme aus. Wir können dann als Entwickler bei der Implementierung einer weiteren Methode entscheiden, ob wir bei Aufruf der ersten Methode eine Ausnahmebehandlung machen, oder die Ausnahme an den Aufrufer unserer zweiten Methode weiterreichen. In späteren Kapiteln werden wir noch genauer diskutieren, welche der beiden Varianten in welchen Fällen geeigneter ist. Bis dahin werden wir das in Übungsaufgaben in der Regel vorgeben.

Einen Sonderfall stellen übrigens Lambda-Ausdrücke dar. Stellen Sie sich vor, dass in einem Lambda-Ausdruck eine geprüfte Ausnahme ausgelöst wird. Nach dem, was Sie in diesem Kapitel über Ausnahmen generell gelernt haben: Kann diese Ausnahme weitergereicht werden, oder muss sie im Lambda-Ausdruck behandelt werden? Und wie ist das mit ungeprüften Ausnahmen?

6.3.3 Eigene Ausnahmeklassen implementieren

Die Ausnahmeklassen aus der API sind in der Regel nicht spezifisch genug für eine bestimmte neue Anwendung. Dadurch entsteht der Bedarf nach leicht modifizierten Ausnahmeklassen für Fehler, die spezifisch nur für die neue Anwendung charakteristisch sind. Solche Fehler können zum Beispiel ungültigen Attributwerte (wie bereits betrachtet) oder Fehler beim Laden oder Speichern von Daten (kommt später noch) sein. Beim Erstellen eigener Ausnahmeklassen achten wir jeweils darauf, dass die Klasse möglichst vielsagend benannt ist und weder zu allgemein noch zu spezifisch ist: Eine zu allgemeine Ausnahmeklasse `SomethingWentWrongException` ist ebenso wenig hilfreich wie eine viel zu spezifische Ausnahmeklasse `RadiusOfCircleWasSetToMinusThreeException`. Wie spezifisch eine Ausnahmeklasse sein darf beziehungsweise muss, hängt natürlich vom jeweiligen Anwendungsgebiet ab, für das die Ausnahme entwickelt wurde. Bei der Entwicklung eigener Ausnahmeklassen wenden wir ein wichtiges Prinzip der Benutzung der API an:

Eigene Klassen lassen sich durch Vererbung aus API-Klassen ableiten.

Dadurch stehen automatisch die Grundfunktionlitäten der API zur Verfügung. Gleichzeitig erlaubt die Vererbung die spezifische Anpassung der eigenen Klasse auf die betrachtete Anwendung.

Wir entwickeln eigene Ausnahmeklassen also **als Unterklassen von API-Ausnahmen**. Das sieht im Allgemeinen wie in Abbildung 2 aus. Dadurch stehen uns alle Grundfunktionalitäten von API-Ausnahmen (wie zum Beispiel Fehlernachrichten) automatisch zur Verfügung und wir können unsere eigene Ausnahmeklasse auf die Anwendung anpassen.

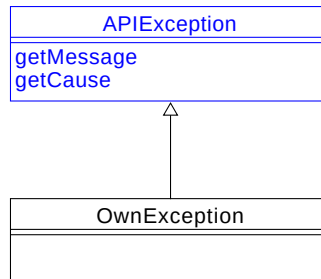


Abbildung 2: Ableitung eigener Ausnahmeklassen aus der API.

Möchte man eine eigene geprüfte Ausnahmeklasse implementieren, so tut man das, indem man die neue Ausnahmeklasse als direkte Unterklasse von `Exception` implementiert (siehe Abbildung 3).

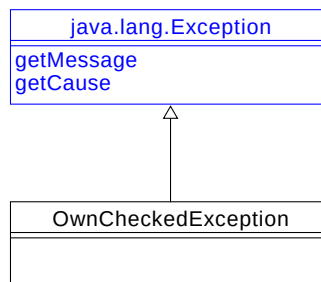


Abbildung 3: Ableitung eigener geprüfter Ausnahmeklassen aus der API.

Dabei sollte man mindestens das Setzen einer individuellen Fehlernachricht ermöglichen, indem man einen entsprechenden Konstruktor definiert. Dieser sollte mit `super` den entsprechenden Oberklassen-Konstruktor aufrufen, welcher die übergebene Fehlernachricht (als String) speichert. Diese kann dann mit `getMessage` abgefragt werden. Das sieht im einfachsten Fall am Beispiel einer allgemeinen Ausnahme für ungültige Attributwerte wie folgt aus:

```
public class IllegalArgumentException extends Exception {
    public IllegalArgumentException(String message) {
        super(message);
    }
}
```

Mehr ist nicht zu tun – wir benutzen nichts weiter als den Oberklassenkonstruktor. Bei Auftreten

der Ausnahme wird an den Konstruktor eine spezifische Fehlermeldung übergeben, zum Beispiel ein Hinweis darauf, welcher Attributwert ungültig ist und warum.

Als Zusatzinformation kann nach demselben Prinzip eine andere Ausnahme `Cause` über den Aufruf des entsprechenden Oberklassen-Konstruktors weitergereicht werden. Wie genau dieser Konstruktor aussieht, entnehmen Sie dem API-Eintrag der Klasse `Exception`. Ein Beispiel dafür besprechen wir in den Kapiteln zum Laden und Speichern von Daten mit Textdateien.

Falls noch weitere spezifische Informationen gespeichert werden sollen, verwendet man dazu Attribute, die über Konstruktorparameter gesetzt und über Get-Methoden abgefragt werden können. Zum Beispiel könnten wir wie folgt unsere eigene Ausnahmeklasse von oben erweitern:

```
public class IllegalArgumentException extends Exception {

    private String attribute;
    private String format;
    private String value;

    public IllegalArgumentException(String message, String attribute,
        String format, String value) {
        super(message);
        this.attribute = attribute;
        this.format = format;
        this.value = value;
    }

    public String getAttribute() {
        return attribute;
    }

    public String getFormat() {
        return format;
    }

    public String getValue() {
        return value;
    }
}
```

Hier werden als zusätzliche Informationen das betroffene Attribut, das geforderte Format für gültige Werte und der ungültige Wert (jeweils als Strings) abgespeichert, und alle drei können bei Bedarf mit einer Get-Methode abgefragt werden.

Hier verzichten wir übrigens üblicherweise auf Set-Methoden, da die Attribute für ein Ausnahmeobjekt nur einmal im Konstruktor gesetzt und danach nicht mehr verändert werden.

Die damit entworfene eigene geprüfte Ausnahmeklasse ist sehr allgemein für verschiedene Attribute einer Datenklasse verwendbar, und kann auch für ungültige Attributwerte in verschiedenen Datenklassen verwendet werden, da das Attribut und das Format für gültige Werte jeweils neu gesetzt werden können.

Stattdessen könnten wir uns auch dafür entscheiden, für jedes Attribut einer Datenklasse eine spezifische Ausnahmeklasse anzulegen, zum Beispiel:

```
public class IllegalNameException extends IllegalArgumentException {  
  
    private static String format = "Namen beginnen mit einem  
        Grossbuchstaben";  
    private String value;  
  
    public IllegalNameException(String message, String value) {  
        super(message);  
        this.value = value;  
    }  
  
    public static String getFormat() {  
        return format;  
    }  
  
    public String getValue() {  
        return value;  
    }  
}
```

Eine eigene geprüfte Ausnahmeklasse sieht allgemein also wie folgt aus:

Definition 6.11: Aufbau eigener geprüfter Ausnahmen

```
public class CheckedException extends Exception {  
  
    /* Optionale Attribute für Zusatzinformationen */  
  
    public CheckedException(String message /*,  
        Optionale Parameter für Zusatzinformationen */  
    ) {  
        super(message); // Fehlernachricht setzen  
        /* Optionale Attribute setzen */  
    }  
  
    /* Optionale Get-Methoden für Zusatzinformationen  
        */  
}
```

Solche eigenen geprüften Ausnahmen verwenden wir zum Beispiel immer dann, wenn der Benutzer über eine freie Eingabe ungültige Werte für ein Attribut übergeben kann. Solche Ausnahmen müssen wir abfangen, da ungültige Werte zu Fehlern in der Benutzung des Programms führen.

Bei nicht freien Eingaben benötigen wir für ungültige Werte nur ungeprüfte Ausnahmen. Das ist zum Beispiel der Fall, wenn es zum Setzen eines Attributwerts ein Auswahlmenü mit festen vorgegebenen Werten gibt, oder Positionen von Mausklicks in eine Zeichenfläche übernommen

werden. Kommt es hier zu einer Ausnahme, deutet das auf einen Programmierfehler bei der Implementierung der grafischen Benutzerschnittstelle hin – ein typischer Anwendungsfall ungeprüfter Ausnahmen.

Um eine eigene ungeprüfte Ausnahmeklasse zu implementieren, lässt man die Klasse (direkt oder indirekt) von `RuntimeException` erben (siehe Abbildung 4).

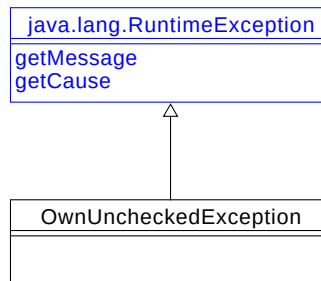


Abbildung 4: Ableitung eigener ungeprüfter Ausnahmeklassen aus der API.

Der Compiler erwartet dann nicht, dass im Methodenkopf angegeben wird, dass diese Ausnahme geworfen werden könnte. Abgesehen von der Oberklasse werden ungeprüfte Ausnahmen aber genauso implementiert wie geprüfte Ausnahmen. Das kann für eine Ausnahme für ungültige negative Werte von Dimensionen einer Form wie der Radius eines Kreises oder die Länge und Breite eines Rechtecks aussehen wie im bereits diskutierten Beispiel `IllegalDimensionException`.

```
public class IllegalDimensionException extends RuntimeException {

    private static String format = "Der Wert muss nicht-negativ sein"
    ;
    private String value;

    public IllegalDimensionException(String message, String value) {
        super(message);
        this.value = value;
    }

    public static String getFormat() {
        return format;
    }

    public String getValue() {
        return value;
    }
}
```

Da der Benutzer solche Dimensionen nicht frei wählen kann (da sie über Interaktionen mit der Maus festgelegt werden), deutet es auf einen Programmierfehler hin, wenn diese Ausnahme geworfen wird. Da bei Programmierfehlern nicht gewährleistet werden kann, dass das Programm

sich noch korrekt verhält, muss diese Ausnahme nicht abgefangen und behandelt werden, sondern sollte direkt zum Abbruch des Programms führen. Eine eigene ungeprüfte Ausnahmeklasse sieht also im Allgemeinen aus wie die eine ungeprüfte Ausnahme – bis auf die Oberklasse, die nun `RuntimeException` ist:

Definition 6.12: Aufbau eigener ungeprüfter Ausnahmen

```
public class UncheckedException extends
    RuntimeException {

    /* Optionale Attribute für Zusatzinformationen */

    public UncheckedException(String message /*,
        Optionale Parameter für Zusatzinformationen */
    ) {
        super(message); // Fehlernachricht setzen
        /* Optionale Attribute setzen */
    }

    /* Optionale Get-Methoden für Zusatzinformationen
        */
}
```